ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



BÁO CÁO TUẦN 1

Đề tài "Khảo sát không gian thiết kế Shared L2 Cache đánh giá bằng gem5"

Team Lead: PhD. Lê Minh Sơn

Mentor: Cao Văn Nam

Họ và tên SV: Lương Khánh Ngọc

MSSV: 24200134

TP. Hồ Chí Minh, tháng 10 năm 2026

Mục lục

I. Giới thiệu **1**

1. Bối cảnh . . . 1
2. Đề tài và mục tiêu . . . 1
3. Kiến trúc hệ thống mục tiêu . . . 2
4. Cấu trúc báo cáo . . . 2

II. Nền tảng kiến thức **3**

1. Các khái niệm và định nghĩa . . . 3
   - 1.1 Cache và các thành phần cơ bản . . . 3
   - 1.2 Cache Line / Block . . . 3
   - 1.3 gem5 và các chế độ mô phỏng . . . 3
   - 1.4 CVA6 và MinorCPU . . . 4
   - 1.5 L2 shared (line 64B, 16 way) . . . 4
   - 1.6 Số way hoạt động và way-shutdown . . . 4
   - 1.7 Dung lượng và associativity . . . 5
   - 1.8 Miss rate, Hit và AMAT . . . 5
   - 1.9 Workload và synthetic traffic generator . . . 6
2. Lý do sử dụng gem5 . . . 7
3. Thời điểm và phạm vi áp dụng của gem5 . . . 7
4. Đối tượng sử dụng . . . 7
5. Giới hạn của phương pháp mô phỏng . . . 7

III. Phương pháp thực hiện Workload **8**

1. Tổng quan vai trò và sản phẩm đầu ra . . . 8
2. Xây dựng và đặc trưng hóa workload . . . 8
3. Thu thập chỉ số và đánh giá hiệu năng . . . 9
4. Tích hợp DSE và phối hợp nhóm . . . 9

IV. Quan hệ phụ thuộc giữa các khâu **9**

1. Sự phụ thuộc của phần Workload vào các khâu khác . . . 9
2. Sự phụ thuộc của các khâu khác vào phần Workload . . . 9

V. Lộ trình thực hiện **10**

I. Giới thiệu

1. Bối cảnh

Các vi xử lý hiện nay hầu hết có nhiều nhân core. Mỗi core thường có cache L1 riêng, nhỏ và rất nhanh. Phía sau L1 là cache L2, có thể dùng riêng hoặc **dùng chung** giữa các core. L2 dùng chung giúp tận dụng dung lượng linh hoạt hơn: core nào đang cần nhiều dữ liệu thì chiếm nhiều chỗ hơn, và các core chia sẻ dữ liệu không phải lưu trùng nhau. Đổi lại, các core phải tranh chấp dung lượng và băng thông của L2.

Cấu hình của L2 (dung lượng, số way, số way đang bật) ảnh hưởng trực tiếp đến hiệu năng, điện năng và diện tích chip. Thử từng cấu hình trên phần cứng hoặc RTL rất tốn thời gian, nên giai đoạn đầu thường dùng mô phỏng kiến trúc. Đề tài này dùng **gem5** cho mục đích đó.

2. Đề tài và mục tiêu

Đề tài: Khảo sát không gian thiết kế L2 chia sẻ bằng gem5.

Mô hình được dựng: hệ thống 4 core RISC-V in-order dùng **MinorCPU**. Nhóm chọn MinorCPU vì nó gần với CVA6 hơn O3CPU. O3CPU là out-of-order nên sinh ra mức memory-level parallelism cao hơn thực tế, làm kết quả lệch so với một core in-order như CVA6.

Tham số cố định: L2 chia sẻ dùng line 64 byte và 16 way, là tham số của bản production.

Các trục được khảo sát (theo thứ tự ưu tiên):

1. **Số way hoạt động**, phục vụ việc đánh giá phương án way-shutdown (mức ưu tiên P1).
2. **Dung lượng** L2.
3. **Associativity.**

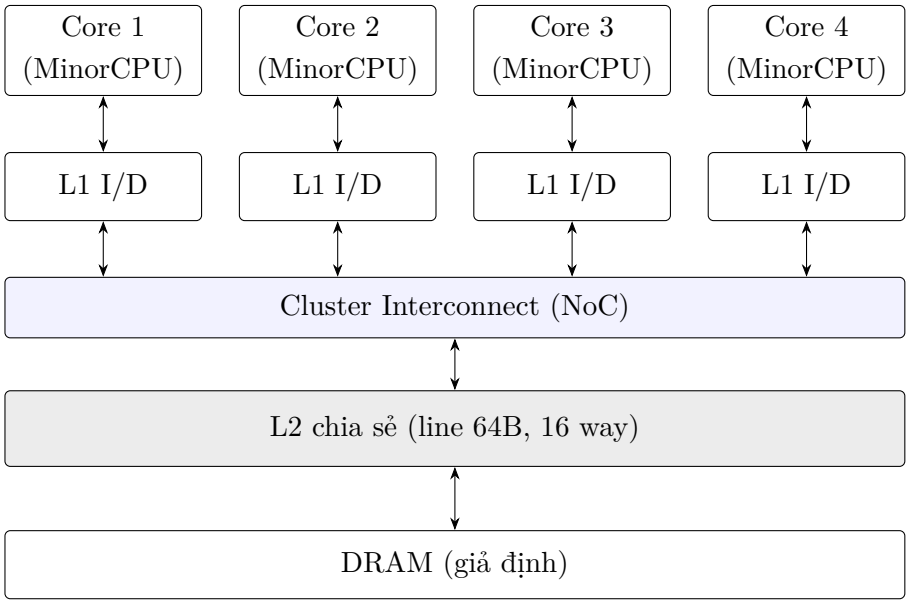
Traffic cần xấp xỉ: traffic của ứng dụng vector/DSP được xấp xỉ bằng một **synthetic traffic generator** có các tham số stride và footprint đặc trưng.

Sản phẩm đầu ra của đề tài:

- Bảng **miss rate** theo từng cấu hình. Đây là số đo đáng tin, vì miss rate do chuỗi địa chỉ truy cập và hình học cache quyết định.
- Bảng **AMAT** (thời gian truy cập bộ nhớ trung bình), kèm các giả định latency của NoC và DRAM đã dùng. Phần latency này là ước lượng theo mô hình.
- Mục **"Giới hạn mô hình"**, nêu rõ phần nào là số đo, phần nào là xấp xỉ, phần nào là tính toán.

3. Kiến trúc hệ thống mục tiêu

Hệ thống gồm bốn core MinorCPU. Mỗi core có L1 instruction và L1 data riêng. Cả bốn core cùng truy cập một L2 chia sẻ (line 64B, 16 way), phía sau là bộ nhớ chính (DRAM), như minh họa ở Hình I.1. Độ trễ của đường truyền giữa các tầng (NoC, DRAM) được xem là **giả định** và dùng khi tính AMAT, không phải là đối tượng khảo sát.



Hình I.1: Kiến trúc hệ thống mục tiêu: 4 core, L1 riêng, kết nối qua NoC tới L2 chia sẻ.

4. Cấu trúc báo cáo

- **Chương I:** Giới thiệu đề tài, mục tiêu và phân công.
- **Chương II:** Nền tảng kiến thức, trình bày theo bốn câu hỏi: là gì, vì sao, khi nào, ai sử dụng.
- **Chương III:** Phần việc của em: generator, workload, đặc trưng hóa, AMAT, giao diện cho DSE.
- **Chương IV:** Quan hệ phụ thuộc giữa các khâu, khâu nào phải xong trước.
- **Chương V:** Lộ trình theo tuần đến 21/12 và các cột mốc.

II. Nền tảng kiến thức

1. Các khái niệm và định nghĩa

1.1 Cache và các thành phần cơ bản

Cache là toàn bộ hệ thống nhớ SRAM tốc độ cao đặt sát CPU, lưu trữ tạm thời các khối dữ liệu thường dùng từ DRAM để CPU không phải chờ đợi lâu. Cấu trúc vật lý của Cache bao gồm:

- **Bank:** Các khối phần cứng độc lập trong Cache, có cổng giải mã và đường đọc/ghi riêng. Nhờ chia Bank, CPU có thể truy xuất song song nhiều dữ liệu cùng lúc mà không bị nghẽn (nếu các dữ liệu nằm ở các Bank khác nhau).
- **Set:** Một hàng dữ liệu trong Bank, được xác định bởi trường Set Index trong địa chỉ CPU phát ra. CPU chỉ được phép tìm dữ liệu trong đúng 1 Set tương ứng với Index đó.
- **Way:** Các ngăn chứa dữ liệu song song nằm trong cùng một Set. Số lượng Way trong 1 Set chính là Associativity.

1.2 Cache Line / Block

Cache Line (hay Block) là đơn vị lưu trữ cơ bản nhỏ nhất trong Cache. Thực chất, mỗi một Way là một Cache Line hoàn chỉnh. Một Cache Line thường gồm 4 thành phần:

- **Valid bit:** 1 bit báo hiệu dữ liệu có hợp lệ hay không (1: có dữ liệu, 0: đang trống).
- **Dirty bit:** 1 bit báo hiệu dữ liệu trong Cache đã bị CPU sửa đổi hay chưa (1: đã sửa, 0: chưa sửa).
- **Tag Store:** Lưu các bit địa chỉ cao để nhận diện khối dữ liệu này thuộc vùng nào trên RAM.
- **Data Block:** Khối dữ liệu thật lấy từ DRAM lên. Với đề tài này, kích thước là 64 byte, được chia thành 16 Words.

1.3 gem5 và các chế độ mô phỏng

gem5 là bộ mô phỏng kiến trúc máy tính mã nguồn mở, viết bằng C++ và Python. Người dùng mô tả hệ thống cần mô phỏng (CPU, cache, bộ nhớ) bằng script Python rồi chạy chương trình lên hệ thống đó. gem5 hỗ trợ nhiều tập lệnh, trong đó có RISC-V. Kết quả mô phỏng được ghi vào tệp thống kê (`stats.txt`) với rất nhiều chỉ số như số truy cập, số miss, số chu kỳ.

1.4 CVA6 và MinorCPU

CVA6 là core RISC-V 64-bit, thiết kế in-order, mã nguồn mở, do cộng đồng OpenHW Group phát triển và có thể chạy Linux. gem5 không có sẵn mô hình riêng cho CVA6, nên phải xấp xỉ bằng một mô hình CPU có sẵn.

Đề tài chọn **MinorCPU** vì đây là mô hình in-order, gần với CVA6 hơn O3CPU. O3CPU là out-of-order: nó cho phép nhiều lệnh truy cập bộ nhớ chờ cùng lúc, tức là mức **memory-level parallelism** cao hơn thực tế của một core in-order. Khi đó độ trễ của các lần miss bị che bớt, và đánh giá về L2 sẽ lạc quan hơn so với CVA6 thật.

MinorCPU vẫn chỉ là xấp xỉ: độ sâu pipeline, bộ dự đoán rẽ nhánh và độ trễ từng tầng không khớp hoàn toàn với CVA6. Điểm này được ghi vào mục giới hạn mô hình (2.5).

1.5 L2 shared (line 64B, 16 way)

L2 chia sẻ là cache cấp hai mà cả bốn core cùng truy cập. Hai tham số cố định của đề tài:

- **Line 64 byte:** đơn vị dữ liệu cache lưu và trao đổi là 64 byte.
- **16 way:** mỗi set chứa 16 line, nên một địa chỉ có thể nằm ở một trong 16 vị trí của set đó.

Dung lượng liên hệ với các tham số theo công thức:

$$\text{dung lượng} = \text{số set} \times \text{số way} \times \text{kích thước line}$$

Ví dụ với L2 dung lượng 1MB, 16 way, line 64B: số set = 1MB / (16 × 64B) = 1024 set, mỗi way có dung lượng 64KB. Một địa chỉ khi vào cache được chia thành 6 bit offset (chọn byte trong line), 10 bit index (chọn set) và phần còn lại là tag.

Vì cả bốn core dùng chung, các core cạnh tranh dung lượng của L2. Đây là lý do cấu hình L2 ảnh hưởng nhiều đến hiệu năng.

1.6 Số way hoạt động và way-shutdown

Way-shutdown là kỹ thuật tắt bớt một số way của cache khi chưa cần dùng hết, nhằm giảm điện năng (đặc biệt là điện năng rò rỉ). Số way còn bật gọi là **số way hoạt động**.

Khi chỉ còn w way hoạt động trong tổng số 16 way:

- Số set giữ nguyên, nhưng associativity giảm còn w.
- Dung lượng hiệu dụng bằng w/16 dung lượng đầy đủ.

Ví dụ: L2 1MB 16 way, nếu chỉ bật 8 way thì còn 512KB hiệu dụng và 8 way mỗi set.

Cái giá phải trả: nếu workload cần nhiều dung lượng hoặc có nhiều địa chỉ rơi vào cùng set, việc tắt way làm miss rate tăng. Ngược lại, với workload nhỏ thì tắt bớt

gần như không ảnh hưởng.

1.7 Dung lượng và associativity

Miss của cache thường được chia thành 4 loại:

- **Compulsory miss:** lần truy cập đầu tiên vào một line.
- **Capacity miss:** cache không đủ chỗ chứa vùng dữ liệu đang dùng.
- **Conflict miss:** nhiều địa chỉ dồn vào cùng một set nên đẩy nhau ra, dù cache vẫn còn chỗ ở set khác.
- **Coherence miss:** do dữ liệu bị vô hiệu hóa bởi core khác (hệ đa lõi).

Tăng **dung lượng** chủ yếu làm giảm capacity miss. Tăng **associativity** chủ yếu làm giảm conflict miss, nhưng làm so sánh tag phức tạp hơn, có thể tăng độ trễ và điện năng.

1.8 Miss rate, Hit và AMAT

Để đánh giá hiệu năng của hệ thống phân cấp bộ nhớ (memory hierarchy performance), thước đo tối ưu thường được dùng là **Thời gian truy cập bộ nhớ trung bình (Average Memory Access Time - AMAT)**.

- **Cache Hit:** Xảy ra khi CPU tìm trong Set tương ứng, có ít nhất một Way chứa Tag trùng khớp với Tag mà CPU yêu cầu, đồng thời Valid bit phải bằng 1.
- **Miss rate (Tỉ lệ Cache Miss):** Là số lần tìm không thấy chia cho tổng số lần truy cập bộ nhớ. Đối với hệ thống cache đa cấp (Multilevel Caches), miss rate được phân biệt thành hai loại:
  - **Local miss rate (Tỉ lệ miss cục bộ):** Số lần miss tại một tầng cache chia cho tổng số lần truy cập vào chính cache đó (ví dụ: $Miss\ rate_{L1}$ hoặc $Miss\ rate_{L2}$).
  - **Global miss rate (Tỉ lệ miss toàn cục):** Số lần miss tại một tầng cache chia cho tổng số lần truy cập bộ nhớ do CPU tạo ra ban đầu (ví dụ: $Miss\ rate_{L1} \times Miss\ rate_{L2}$).
- **AMAT** được tính theo công thức cơ bản: $AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$.

Áp dụng cho hệ thống có hai tầng cache (L1 và L2 chia sẻ), công thức được mở rộng thành:

$$AMAT = Hit\ time_{L1} + Miss\ rate_{L1} \times (Hit\ time_{L2} + Miss\ rate_{L2} \times Miss\ penalty_{L2})$$

Trong mô hình của đề tài, các thông số $Hit\ time$ tương ứng với thời gian truy cập nội bộ của L1 và L2, trong khi $Miss\ penalty_{L2}$ sẽ cộng gộp độ trễ giả định của đường truyền NoC và bộ nhớ chính (DRAM). Vì AMAT là một chỉ số mang tính **tính**

toán xấp xỉ, mọi báo cáo hiệu năng dựa trên AMAT đều phải đi kèm các cấu hình giả định về độ trễ. Quá trình DSE nhằm cải thiện AMAT sẽ tập trung vào ba hướng tối ưu cơ bản: giảm Miss Rate, giảm Miss Penalty và giảm Hit Time.

Goals	Basic Approaches
Reducing Miss Rate	Larger block size, larger cache size and higher associativity
Reducing Miss Penalty	Multilevel caches, and higher read priority over writes
Reducing Hit Time	Avoid address translation when indexing the cache

Hình II.1: Các hướng tiếp cận cơ bản để tối ưu hóa hiệu năng Cache.

1.9 Workload và synthetic traffic generator

- **Workload** là chương trình được chạy trên hệ thống mô phỏng, ví dụ các kernel xử lý tín hiệu hoặc vector.
- **Synthetic traffic generator** là bộ sinh chuỗi truy cập bộ nhớ theo các tham số cho trước, không thực hiện tính toán thật. Hai tham số chính:
  - **Stride:** khoảng cách giữa hai địa chỉ truy cập liên tiếp.
  - **Footprint:** tổng kích thước vùng nhớ được truy cập khác nhau mà workload chạm tới, tức là không tính truy cập lặp lại. Duyệt một mảng 1MB mười lượt thì footprint vẫn là 1MB, không phải 10MB.

Ứng dụng vector/DSP thường duyệt các mảng lớn theo bước đều và ít tái sử dụng dữ liệu, nên có thể xấp xỉ bằng generator có stride và footprint phù hợp.

Hành vi miss tương ứng với các tham số:

Điều kiện	**Kết quả mong đợi**
Footprint nhỏ hơn dung lượng hiệu dụng của L2	Sau giai đoạn khởi động, hầu hết là hit.
Footprint lớn hơn dung lượng hiệu dụng	Capacity miss tăng.
Stride nhỏ hơn 64B	Nhiều truy cập liên tiếp trúng cùng một line, nên miss ít.
Stride bằng bội của (số set × 64B)	Các địa chỉ dồn vào cùng một set, gây conflict miss.

Đây cũng là lý do generator hữu ích: nó cho phép điều khiển chính xác từng yếu tố gây miss, điều mà chương trình thật khó làm được.

2. Lý do sử dụng gem5

- **Tốc độ mô phỏng:** gem5 chạy nhanh hơn khoảng 1000 lần so với mô phỏng mức RTL (Verilog), cho phép kiểm tra và tinh chỉnh "chip ảo"ngay trên máy tính cá nhân trong vài phút trước khi bắt tay vào làm phần cứng thật.
- **Tính linh hoạt:** Thay đổi tham số hệ thống (chẳng hạn như cấu hình cache, số core) cực kỳ dễ dàng thông qua các tệp kịch bản (script) bằng Python, không đòi hỏi phải viết lại mã nguồn phần cứng phức tạp.
- **Chiết xuất thống kê nội bộ chi tiết:** Công cụ cho phép ghi nhận toàn diện các thông số từ `stats.txt`, dễ dàng theo dõi số lần truy cập, miss rate ở từng cấp độ cache và hỗ trợ tính toán AMAT.

3. Thời điểm và phạm vi áp dụng của gem5

- **Giai đoạn thăm dò kiến trúc (Architecture Exploration):** Mô phỏng gem5 nằm ở bước sơ khai nhất của toàn bộ quy trình thiết kế chip, thực hiện ngay cả trước khi chốt đặc tả vi kiến trúc và viết mã phần cứng RTL.
- **Phạm vi áp dụng:** Rất phù hợp để đánh giá và định hướng các quyết định lớn ở mức hệ thống (System-level), chẳng hạn như tìm ra dung lượng L2 cần thiết (Capacity Scaling), độ kết hợp phù hợp (Associativity), hay mô phỏng mức tiêu thụ năng lượng khi áp dụng kỹ thuật tắt way (Way-shutdown).

4. Đối tượng sử dụng

Vị trí	**Mục đích sử dụng**
Kỹ sư kiến trúc / mô hình hiệu năng	Quét cấu hình, chọn tham số cache, phân tích trade-off.
Kỹ sư thiết kế vi kiến trúc / RTL	Dùng kết quả mô phỏng làm cơ sở cho thiết kế cache.
Kỹ sư phân tích hiệu năng (workload)	Chọn, chạy và đặc trưng hóa benchmark, xây generator.
Kỹ sư công suất / năng lượng	Dùng kết quả về way-shutdown để đánh giá tiết kiệm điện.
Nhà nghiên cứu, sinh viên	Thử ý tưởng mới trước khi hiện thực.

5. Giới hạn của phương pháp mô phỏng

- MinorCPU chỉ **xấp xỉ** CVA6, không khớp hoàn toàn về pipeline.

- Generator chỉ **xấp xỉ** traffic vector/DSP thật, nên cần kiểm chứng bằng workload thật.
- Độ trễ NoC và DRAM là **giả định** ước lượng, không đo từ chip thật.
- gem5 là mô hình, không chính xác từng chu kỳ như chip thật, nên kết quả dùng chủ yếu để **so sánh tương đối** giữa các cấu hình.
- Miss rate là số đo, còn AMAT là kết quả tính toán dựa trên giả định. Hai loại này không được trình bày lẫn lộn (xem mục 3.7).

III. Phương pháp thực hiện phần Workload và Synthetic Traffic

Chương này trình bày chi tiết quy trình thực hiện phần việc Workload và Synthetic Traffic dưới vai trò System Architect & Performance Modeling Engineer, từ khâu xác định đầu ra, xây dựng công cụ, đo lường chỉ số đến việc phối hợp tích hợp với các khâu khác trong nhóm.

1. Tổng quan vai trò và sản phẩm đầu ra

Nhiệm vụ chính là kiểm soát và cung cấp toàn bộ "đầu vào" cho hệ thống mô phỏng gem5. Việc đánh giá L2 chỉ chính xác khi workload phản ánh đúng hành vi của ứng dụng Vector/DSP thực tế.

Sản phẩm đầu ra bao gồm: danh sách workload, cấu hình generator, báo cáo kiểm chứng, mô hình latency NoC

2. Xây dựng và đặc trưng hóa workload

- **Sử dụng Synthetic Traffic:** Thiết kế các luồng truy cập nhân tạo để chủ động thay đổi tham số bước nhảy (*stride*) và không gian nhớ (*footprint*). Việc này tạo ra một ma trận thí nghiệm đa dạng để quét (sweep) qua các giới hạn của L2, trả lời cho câu hỏi về Capacity Scaling và Associativity.
- **Đối chiếu bằng Workload thật:** Các ứng dụng thực tế (kernel xử lý tín hiệu, tính toán vector) được đưa vào mô phỏng để kiểm chứng lại tính chính xác của Synthetic Traffic.
- **Kịch bản Way-shutdown (P1):** Phân loại workload theo mức độ sử dụng cache để tạo ra các kịch bản tải nhẹ (light workloads). Từ đó đánh giá khả năng tắt bớt các way không dùng đến, nhằm cắt giảm năng lượng rò rỉ (static leakage power) theo đúng ưu tiên P1 của dự án.

3. Thu thập chỉ số và đánh giá hiệu năng

- **Trích xuất Miss rate:** Lấy số liệu miss rate của L1 và L2 từ `stats.txt` tương ứng với từng cấu hình.
- **Tính toán AMAT:** Áp dụng miss rate thu được vào công thức tính $AMAT$, kết hợp với các độ trễ giả định của NoC và DRAM.
- **Phân loại kết quả:** Rạch ròi giữa số đo (miss rate từ gem5), số xấp xỉ (đặc tính traffic) và số tính toán (AMAT), giúp giới hạn rõ phạm vi sai số của mô hình.

4. Tích hợp DSE và phối hợp nhóm

- **Chuẩn hóa giao diện:** Các thông số đầu vào và định dạng đầu ra được quy chuẩn hóa để tự động hóa quét cấu hình (DSE Automation).
- **Phối hợp nền tảng:** Workload được chạy trên base CVA6 (4 core + NoC) đã được thiết lập.
- **Kiểm chứng Way-shutdown:** Cung cấp các kịch bản workload để kiểm tra tính đúng đắn của logic tắt way trên gem5.

IV. Quan hệ phụ thuộc giữa các khâu

1. Sự phụ thuộc của phần Workload vào các khâu khác

Để có thể chạy và đặc trưng hóa workload thành công, cần nhận được các thành quả sau:

- **Cấu hình base CVA6:** Không thể chạy workload nếu nền tảng 4 core MinorCPU + L1 + NoC chưa được dựng xong và cấu hình đúng trong gem5.
- **Cơ chế Way-shutdown:** Quá trình đánh giá hiệu năng (miss rate) theo số way hoạt động chỉ có thể thực hiện khi đã hiện thực xong logic tắt/mở way trên cấu hình L2 trong gem5.

2. Sự phụ thuộc của các khâu khác vào phần Workload

Ngược lại, phần việc này đóng vai trò là "nhiên liệu"(đầu vào) không thể thiếu cho các phần khác:

- **Bàn giao cho phần L2 Cache:** Cần bộ Synthetic Traffic Generator và các kịch bản workload có footprint khác nhau để kiểm tra xem thuật toán Way-shutdown có hoạt động đúng thiết kế và mang lại hiệu quả tiết kiệm năng lượng thật sự hay không.

– **Phục vụ DSE Automation:** Để viết script chạy tự động hàng ngàn cấu hình, cần bàn giao sớm: (1) Cú pháp lệnh gọi workload, (2) Quy chuẩn định dạng file tham số đầu vào, và (3) Vị trí các thông số cần trích xuất (miss rate) trong tệp `stats.txt`. Nếu chậm trễ khâu này, pipeline tự động sẽ bị đình trệ.

V. Lộ trình thực hiện

Tuần 1 – Tuần 3: Tìm hiểu và chuẩn bị workload, benchmark, memory access pattern trên gem5 để mô phỏng Vector/DSP traffic.

Tuần 4 – Tuần 5: Xây dựng và kiểm thử synthetic traffic, tạo các kịch bản light workloads phục vụ kiểm chứng Way-shutdown (P1).

Tuần 6: Hoàn thiện ma trận thí nghiệm workload, script và tài liệu → **bàn giao cho thành viên khác** để bắt đầu chạy quét DSE trên Shared L2.

Tuần 7 – Tuần 12: Hỗ trợ team tích hợp, thu thập L2 Miss Rate Matrix, tính toán AMAT và đưa ra Hardware Specification cho phần RTL.

Trên đây chỉ là lộ trình dự kiến, có thể thay đổi và điều chỉnh trong quá trình làm cho phù hợp.